

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Title

Tesi di Laurea Magistrale in
Ingegneria Informatica
Computer Science and Engineering

Author: **Domenico Cacace**

Student ID: 977835
Advisor: Prof. Alessandro Barenghi
Co-advisors: Prof. Gerardo Pelosi
Academic Year: 2022-23

Abstract

Abstract EN

Keywords: here, the keywords, of your thesis

Introduzione

Abstract IT

Parole chiave: qui, vanno, le parole chiave, della tesi

Contents

Abstract **i**

Introduzione **iii**

Contents **v**

1 Introduction **1**

2 Background **3**

2.1 Abstract Algebra . . . 3
2.1.1 Field theory . . . 3
2.1.2 Frobenius automorphism . . . 5
2.1.3 Notation . . . 5
2.2 Polynomial multplication . . . 6
2.2.1 Karatsuba multiplication . . . 6
2.2.2 Fast Fourier Transform . . . 8
2.2.3 Additive FFT . . . 10
2.3 Coding Theory . . . 16
2.3.1 General definitions . . . 16
2.3.2 Code families . . . 18
2.3.3 Difficult problems for cryptography . . . 20
2.4 Side Channel Attacks . . . 21
2.4.1 Timing attacks . . . 21
2.4.2 Power attacks . . . 22
2.4.3 Countermeasures . . . 24
2.5 Hamming Quasi-Cyclic . . . 25
2.5.1 Encryption scheme . . . 25
2.5.2 Public Key Encryption . . . 26
2.5.3 Key Encapsulation Mechanism . . . 26

2.5.4 Parameters 27

3 Implementation **29**
3.1 Polynomial multiplication 29
3.1.1 Shift and Add 30
3.1.2 Constant-time shift and add 30
3.1.3 Frobenius Additive FFT 31
3.2 Masking 34
3.2.1 Keygen 34
3.2.2 Encrypt 34
3.2.3 Decrypt 35

4 Empirical Evaluations **37**
4.1 Execution time 38
4.2 Power measurements 40

5 Conclusions **41**

Bibliography **43**

A Appendix A: execution times **47**

List of Figures **49**

List of Tables **51**

List of Algorithms **53**

Listings **55**

Acknowledgements **57**

1 | Introduction

2 | Background

2.1. Abstract Algebra

2.1.1. Field theory

Definition 2.1.1 (Group)**.** Let G be a set. The structure $(G, \cdot)$ is a *group* if it satisfies the following axioms:

- Closure: For all $a, b \in G$, $a \cdot b \in G$.
- Associativity: For all $a, b, c \in G$, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
- Identity: There exists an element $e \in G$ such that for all $a \in G$, $e \cdot a = a \cdot e = a$.
- Inverse: For all $a \in G$, there exists an element $a^{-1} \in G$ such that $a \cdot a^{-1} = a^{-1} \cdot a = e$.

If the group also satisfies the commutative property (i.e. $a \cdot b = b \cdot a$ for all $a, b \in G$), then it is called an *abelian group*.

Commonly, we call $(G, +)$ an *additive* group and $(G, \cdot)$ a *multiplicative* group. In this case, the identity element are denoted by 0 and 1, respectively.

Definition 2.1.2 (Ring)**.** Let R be a set. The structure $(R, +, \cdot)$ is a *ring* if it satisfies the following properties:

- Additive group: $(R, +)$ is an abelian group.
- Associativity w.r.t. multiplication: For all $a, b, c \in R$, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
- Distributivity: For all $a, b, c \in R$, $a \cdot (b + c) = a \cdot b + a \cdot c$ and $(a + b) \cdot c = a \cdot c + b \cdot c$.

Definition 2.1.3 (Field)**.** Let K be a set. The structure $(K, +, \cdot)$ is a *field* if it satisfies the following properties:

- Additive group: $(K, +)$ is an abelian group.
- Multiplicative group: $(K \setminus \{0\}, \cdot)$ is an abelian group.
- Distributivity: For all $a, b, c \in K$, $a \cdot (b + c) = a \cdot b + a \cdot c$ and $(a + b) \cdot c = a \cdot c + b \cdot c$.

Any field is also a ring, but the converse is not necessarily true.

Definition 2.1.4 (Characteristic)**.** Let K be a field. The *characteristic* of K is the smallest positive integer n such that

$$\underbrace{1 + \cdots + 1}_{n \text{ times}} = 0$$

Definition 2.1.5 (Order)**.** Let K be a field. The *order* of K is the amount of elements in K; if the order of K is finite, it is also called a Galois field. A Galois field of order q, denoted as $\mathbb{F}_q$ or $GF(q)$, exists if and only if $q = p^n$ for some prime p; all Galois fields of order q are isomorphic.

Galois fields can be constructed in two ways:

- Primitive fields: $GF(p) \cong \mathbb{Z}_p$, with p prime, is a field of order and characteristic p.
- $GF(p^n) = GF(p)[X]/P(X)$ with $P(X)$ irreducible polynomial in $GF(p)[X]$ of degree n, is a field of order p^n and characteristic p.

Definition 2.1.6 (Primitive element)**.** Let $GF(q)$ be a Galois field. An element $a \in GF(q)$ is a *primitive element* if every nonzero element of $GF(q)$ is a power of a; in other words, a is a generator of the multiplicative group of $GF(q)$. Since the multiplicative group of $GF(q)$ has order $q - 1$, primitive elements are also called $(q - 1)$-th roots of unity.

Definition 2.1.7 (Field extension)**.** Let K and L be fields. If $K \subseteq L$, then we say that K is a *subfield* of L; conversely, we say that L is a *field extension* of K. Given a field extension, denoted as L/K, the field L is a vector space over K; the dimension of the vector space defines the *degree of the extension*, and is denoted by $[L : K]$. In the case of finite fields, $[GF(p^n) : GF(p)] = n$.

Example: Binary fields

The smallest Galois field is $GF(2)$, which is isomorphic to $\mathbb{Z}_2 = \{0, 1\}$. We can build $\mathbb{F}_8 = GF(2^3)$ as $GF(2)[X]/\langle X^3 + X + 1\rangle$. The elements of $\mathbb{F}_8$ are then the binary polynomials of degree strictly less than $n = 3$; to avoid confusion, the elements are written as polynomials in the variable α, where $P(\alpha) = 0$.

We have that $\mathbb{F}_8 = \{0, 1, \alpha, 1 + \alpha, \alpha^2, 1 + \alpha^2, \alpha + \alpha^2, 1 + \alpha + \alpha^2\}$, so the order of the field is 8. Moreover, since $1 + 1 = 0$ in $GF(2)$, we have that $1 + 1 = 0$ in $\mathbb{F}_8$ as well, so the characteristic of $\mathbb{F}_8$ is 2.

By construction, $\mathbb{F}_8$ is a field extension of $GF(2)$ of degree 3; the vector space $\mathbb{F}_8$ is

thus a vector space over $GF(2)$ of dimension 3, having as basis the elements $\{1, \alpha, \alpha^2\}$.

2.1.2. Frobenius automorphism

Definition 2.1.8 (Frobenius map)**.** Let $\mathbb{F}_2$ be the binary field. The Frobenius map is the automorphism defined as:

$$\phi_2 : \mathbb{F}_2 \to \mathbb{F}_2, \quad \phi_2(x) = x^2 \tag{2.1}$$

Let Σ be a set; with the notation $\phi_2(\Sigma)$ we denote the set built as $\{\phi_2(x) \mid x \in \Sigma\}$; moreover, with the notation $\phi_2^{\circ j}$ we denote the function applying the Frobenius map j times.

Definition 2.1.9 (Order of ϕ_2)**.** Let Σ be a set. The order of ϕ_2 on Σ, denoted as $Ord_{\phi_2}(\Sigma)$ is the smallest positive integer j such that $\phi_2^{\circ j}(\Sigma) = \Sigma$.

Definition 2.1.10 (Frobenius partition)**.** Given the sets Σ, Ω such that $Ord_{\phi_2}(\Sigma) = j$, we say that Σ is a Frobenius partition of Ω if

$$\Omega = \Sigma \cup \phi_2(\Sigma) \cup \cdots \cup \phi_2^{\circ(j-1)}(\Sigma) \tag{2.2}$$

and all the sets $\phi_2^{\circ i}(\Sigma)$ are pairwise disjoint.

2.1.3. Notation

Throughout this document, we will use the following algebraic notation:

- $\mathbb{Z}$: the ring of integers.
- $\mathbb{F}_2$: the binary Galois field $GF(2)$.
- $\mathbb{F}_{2^n}$: the Galois field $GF(2^n)$.
- $\mathcal{V} = \mathbb{F}_2^n$: the vector space of dimension n over $GF(2)$, with canonical basis $\{1, \alpha, \cdots, \alpha^{n-1}\}$.
- $\mathcal{R} = \mathbb{F}_2[X]/\langle X^n - 1\rangle$: the ring of binary polynomials of degree less than n.
- $\mathtt{wt}(\cdot)$: the Hamming weight of a vector *i.e.* the number of its non-zero coordinates.

2.2. Polynomial multplication

Let $a(x), b(x) \in \mathcal{R}$ be two polynomials. To calculate the product $c(x) = a(x)b(x)$ the most straightforward approach, also known as the *schoolbook* algorithm, requires us to multiply each coefficient of the first with each polynomial of the second and add together the results of the same degree. With this method we can calculate the product of the polynomials with $\mathcal{O}(n^2)$ multiplications and $\mathcal{O}(n^2)$ additions, as the degree of the operands is at most $n-1$. Even though this approach is simple to understand and implement, it does not scale well as the degree of the operands increases: for this reason, other multiplication algorithms have been proposed.

As shown by Bernstein in [Ber01], we can think of most of the polynomial multiplication algorithms in the literature as follows:

- **Lifting** the operands to a different ring via a homomorphism f
- Multiplying the lifted polynomials in the new ring
- **Mapping** the result back to the original ring via the inverse homomorphism f^{-1}

By carefully choosing the destination ring and the homomorphism, we can obtain algorithms with drastically better asymptotic complexity: in the remainder of this section we will discuss two of the most common algorithms, Karatsuba and the Fast Fourier Transform.

2.2.1. Karatsuba multiplication

The Karatsuba algorithm [KO62] is a divide and conquer algorithm that splits a n-degree polynomial into three $n/2$-degree polynomials and then calculates the product of the three polynomials in $\mathcal{O}(n^{\log_2 3})$ operations.

Starting from $a(x)$, $b(x)$ of degree n and an integer $m < n$, we can write

$$a(x) = a_0(x) + x^m a_1(x)$$
$$b(x) = b_0(x) + x^m b_1(x)$$

where $a_0(x)$ and $b_0(x)$ have degree less than m. At this point, we can calculate the product

$c(x) = a(x) \cdot b(x)$ as

$$\begin{aligned} c(x) &= a(x)b(x) \\ &= (a_0(x) + x^m a_1(x))(b_0(x) + x^m b_1(x)) \\ &= a_1(x)b_1(x)x^{2m} + (a_1(x)b_0(x) + a_0(x)b_1(x))x^m + a_0(x)b_0(x) \\ &= z_2(x)x^{2m} + z_1(x)x^m + z_0(x) \end{aligned}$$

where

$$\begin{aligned} z_2(x) &= a_1(x)b_1(x) \\ z_1(x) &= a_1(x)b_0(x) + a_0(x)b_1(x) \\ z_0(x) &= a_0(x)b_0(x) \end{aligned}$$

Karatsuba observed that, at the cost of some few additions, we can calculate $a(x)b(x)$ with three multiplications, as we can rewrite z_1 as

$$z_1(x) = (a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$$

Example:
From the ring of binary polynomials $\mathcal{R}$ we take $a(x) = 1 + x + x^3$ and $b(x) = x + x^2$; by taking $m = 2$, we rewrite:

$$\begin{aligned} a(x) &= (1 + x) + x^2(x) \\ b(x) &= (x) + x^2(1) \end{aligned}$$

and calculate the subproducts as

$$\begin{aligned} z_2(x) &= (1 + x)(x) = x + x^2 \\ z_0(x) &= (x)(1) = x \\ z_1(x) &= (1 + x + x)(x + 1) - (x) - (x + x^2) = 1 + x + x^2 \end{aligned}$$

From the above, we can calculate the full product as

$$\begin{aligned} c(x) &= (x)x^{2\cdot 2} + (1 + x + x^2)x^2 + (x + x^2) \\ &= x^5 + x^4 + x^3 + x \end{aligned}$$

The input polynomials can be split into an arbitrary amount of parts, and the product calculated in a similar fashion: such a product is calculated by the Toom-Cook algo-

rithm [CA69], which is a generalization of Karatsuba's algorithm.

2.2.2. Fast Fourier Transform

We can approach polynomial multiplication in a totally different way: instead of multiplying the polynomial coefficients, we can evaluate the inputs at a set of points (*sampling*), calculate the pointwise product of the evaluations and then interpolate the points to obtain the product of the polynomials.

Example:
Let $a(x) = x^2 + 2x + 3$ and $b(x) = 4x + 5$ be polynomials with real coefficients of degree at most $n = 2$; since their product will have degree at most $2n = 4$ we evaluate the polynomials at $2n + 1$ points:

$$\begin{aligned} a(0) &= 1 \cdot 0^2 + 2 \cdot 0 + 3 = 3 & b(0) &= 0 \cdot 0^2 + 4 \cdot 0 + 5 = 5 \\ a(1) &= 1 \cdot 1^2 + 2 \cdot 1 + 3 = 6 & b(1) &= 0 \cdot 1^2 + 4 \cdot 1 + 5 = 9 \\ a(2) &= 1 \cdot 2^2 + 2 \cdot 2 + 3 = 11 & b(2) &= 0 \cdot 2^2 + 4 \cdot 2 + 5 = 13 \\ a(3) &= 1 \cdot 3^2 + 2 \cdot 3 + 3 = 18 & b(3) &= 0 \cdot 3^2 + 4 \cdot 3 + 5 = 17 \\ a(4) &= 1 \cdot 4^2 + 2 \cdot 4 + 3 = 27 & b(4) &= 0 \cdot 4^2 + 4 \cdot 4 + 5 = 21 \end{aligned}$$

We can then calulate the pointwise multiplication of the samples

$$\begin{aligned} c(0) &= a(0) \cdot b(0) = 3 \cdot 5 = 15 \\ c(1) &= a(1) \cdot b(1) = 6 \cdot 9 = 54 \\ c(2) &= a(2) \cdot b(2) = 11 \cdot 13 = 143 \\ c(3) &= a(3) \cdot b(3) = 18 \cdot 17 = 306 \\ c(4) &= a(4) \cdot b(4) = 27 \cdot 21 = 567 \end{aligned}$$

Finally, we can interpolate the result to obtain the coefficients of the product:

$$c(x) = 4x^3 + 13x^2 + 22x + 15$$

This approach might appear slower than the schoolbook method: the sampling requires $\mathcal{O}(n^2)$ operations, the pointwise product $\mathcal{O}(n)$ and the interpolation up to $\mathcal{O}(n^3)$ if we use a naive method such as Gaussian elimination; however, with the right choice of sampling

points, we can drastically reduce the operations required in all three steps.

Instead of sampling at $2n$ random points, we can choose the points to be the N-th roots of unity, *i.e.* we calculate the Discrete Fourier Transform of the input polynomials:

$$\mathcal{F}(x) = \sum_{n=0}^{N} x_n e^{-\frac{i2\pi}{N}kn} \tag{2.3}$$

At this point we can perform the pointwise product in the *transformed* domain and, since $\mathcal{F}$ is invertible, apply the inverse transform:

$$\mathcal{F}^{-1}(X) = \frac{1}{N}\sum_{n=0}^{N} X_n e^{i\frac{2\pi}{N}kn} \tag{2.4}$$

The algorithms that compute the discrete Fourier Transform of a sequence (or its inverse) are called fast Fourier transforms (FFT), and have an asymptotic complexity of $\mathcal{O}(n \log n)$.

Cooley-Tukey FFT

The Cooley-Tukey FFT [CT65] is the most common FFT algorithm. It is a *divide et impera* algorithm, which recursively expresses a DFT of size $N = N_1 N_2$ into N_1 smaller DFTs of size N_2 each. The simplest form of the Cooley-Tukey algorithm is the radix-2 decimation-in-time FFT, which computes the DFTs of the even and odd indexed inputs separately and then combines the results. Assuming that N is a power of 2, we can rewrite the DFT as:

$$\begin{aligned}
X_k &= \sum_{n=0}^{N-1} x_n e^{-\frac{2\pi i}{N}nk} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N}(2m)k} + \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N}(2m+1)k} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2}mk} + e^{-\frac{2\pi i}{N}k} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2}mk} \\
&= E_k + e^{-\frac{2\pi i}{N}k} O_k \qquad \text{for } k = 0, \cdots, N/2-1
\end{aligned}$$

where we split the DFT into two smaller DFTs, on the even and odd indexes respectively. Because of the periodicity of the complex exponential, we can also write:

$$\begin{aligned}
X_{k+N/2} &= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} m(k+N/2)} + e^{-\frac{2\pi i}{N}(k+N/2)} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} m(k+N/2)} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} mk} e^{-2\pi mi} + e^{-\frac{2\pi i}{N} k} e^{-\pi i} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} mk} e^{-2\pi mi} \\
&= \sum_{m=0}^{N/2-1} x_{2m} e^{-\frac{2\pi i}{N/2} mk} - e^{-\frac{2\pi i}{N} k} \sum_{m=0}^{N/2-1} x_{2m+1} e^{-\frac{2\pi i}{N/2} mk} \\
&= E_k - e^{-\frac{2\pi i}{N} k} O_k \qquad \text{for } k = 0, \cdots, N/2-1
\end{aligned}$$

This means that we can express a DFT of size N as two DFTs of size $N/2$, where we can re-use intermediate results to calculate multiple outputs. Since the base case of the recursion has complexity $\mathcal{O}(n)$ we get the recurrence relation $T(n) = 2T(n/2) + \mathcal{O}(n)$; following the Master theorem [BHS80], the computational complexity of the Cooley-Tukey FFT is $\mathcal{O}(n \log n)$.

2.2.3. Additive FFT

The Cooley-Tukey FFT works when the n points to sample are n-th roots of unity, and hence form a cyclic multiplicative group; this and similar algorithms are also referred to as *multiplicative* FFTs, as they are based on the factorization of $x^n - 1$.
Unfortunately, multiplicative FFTs do not work when the underlying field does not admit primitive roots of unity, as it happens for fields of characteristic two; for this reason, *additive* FFTs were invented [WZ88, Can89]. Instead of using the n-th roots of unity, additive FFTs evaluate the polynomials at the powers of a primitive $(n-1)$-th root of unity, as well as the zero element. Some examples of such algorithms that work on binary fields are the Gao-Mateer FFT [GM10] and the Lin-Chung-Han FFT [LCH14].
We will now focus on the LCH FFT, as it will be the foundation for a faster approach based on the Frobenius map; for this reason, we will consider the case $n = 2^m$ where m is a power of 2.

Cantor basis

In [Can89] Cantor showed how to get a special basis in which it is possible to compute the FFT efficiently for any $m = p^n$, with p a prime number; later in [GM10] Gao and Mateer used a simpler construction for the special case $p = 2$, where the Cantor basis

$\mathbf{v} = (v_1, \cdots, v_m)$ of $\mathbb{F}_2^m$ follows the structure:

$$v_1 = 1, \quad v_i^2 + v_i = v_{i-1} \text{ for } i > 1 \tag{2.5}$$

With respect to the Cantor basis $\mathbf{v}$, we define its sequence of subspaces as:

$$W_0 := \{0\}, \quad W_i := span\{v_0, \cdots, v_{i-1}\} \text{ for } i \geqslant 1 \tag{2.6}$$

By construction, we have that $W_0 \subset W_1 \subset \cdots \subset W_{m-1} = \mathbb{F}_{2^m}$ and the basis for a smaller space is contained into the basis of a larger space: this means that we can arbitrarily treat elements of smaller fields as elements in the larger fields by padding zeros to the extra dimensions.

With respect to each element of the basis v_i we can define its subspace polynomial $s_i(x)$, also called vanishing polynomial, as:

$$s_i(x) = \prod_{a \in W_i} (x - a) \tag{2.7}$$

Since $\dim(W_i) = i$ we have $\deg(s_i(x)) = |W_i| = 2^i$. Such polynomials, as shown in [Can89, GM10], satisfy the following useful properties:

- linearity: $s_i(x + y) = s_i(x) + s_i(y)$
- recursivity: $s_{i+j}(x) = s_i(s_j(x))$ or, equivalently, $s_{i+1}(x) = s_i(x)^2 + s_i(x)$
- $s_i(v_i) = 1$, for v_i in the Cantor basis
- the roots of $s_i(x)$ are the elements of W_i

Note that the W_is are also fields whenever i is a power of two; in that case, the vanishing polynomial has the form $s_i(x) = x^{2^i} + x$.

> **Example: Cantor basis in $\mathbb{F}_{16}$**
> Recalling that $GF(16) = GF(2)[X]/\langle X^4 + X + 1\rangle$ and that $v_1 = 1$, we can compute $v_i = \beta(1) + \gamma(x) + \delta(x^2) + \epsilon(x^3)$, where the coefficients can be found by solving:
>
> $$\beta(1 + 1^2) + \gamma(x + x^2) + \delta(x^2 + (x^2)^2) + \epsilon(x^3 + (x^3)^2) = v_{i-1}$$
>
> The resulting Cantor basis is $\mathbf{v} = (1, \alpha+\alpha^2, \alpha, \alpha+\alpha^3)$. As a consequence, the subspaces W_i are:
>
> - $W_0 = \{0\}$

- $W_1 = \{0, 1\}$
- $W_2 = \{0, 1, \alpha + \alpha^2, 1 + \alpha + \alpha^2\}$
- $W_3 = \{0, 1, \alpha + \alpha^2, 1 + \alpha + \alpha^2, \alpha, 1 + \alpha, \alpha^2, 1 + \alpha^2\}$
- $W_4 = \{0, 1, \alpha + \alpha^2, 1 + \alpha + \alpha^2, \alpha, 1 + \alpha, \alpha^2, 1 + \alpha^2, \alpha + \alpha^3, 1 + \alpha + \alpha^3, \alpha^2 + \alpha^3, 1 + \alpha^2 + \alpha^3, \alpha^3, 1 + \alpha^3, \alpha + \alpha^2 + \alpha^3, 1 + \alpha + \alpha^2 + \alpha^3\} = \mathbb{F}_{16}$

Consequently, the vanishing polynomials are:

- $s_0(x) = x$
- $s_1(x) = x + x^2$
- $s_2(x) = x^4 + x$
- $s_3(x) = x^8 + x^4 + x^2 + x$
- $s_4(x) = x^{16} + x$

A *novel* polynomial basis In order to perform the LCH FFT, the input polynomial has to be represented in a different basis. Given the Cantor basis $\mathbf{v} = (v_1, \cdots, v_m)$, we define the *novelpoly* basis with respect to $\mathbf{v}$ as $(X_1, \cdots, X_{n-1})$, defined as follows:

$$X_k(x) = \prod (s_i(x))^{b_i} \qquad \text{where } k = \sum_{i=0}^{\log_2 n} b_i 2^i \tag{2.8}$$

In other words, $X_k(x)$ is the product of all the i-th vanishing polynomials, where the is are the set bits of k in its binary representation.

Algorithm 1: Basis conversion from monomial to *novelpoly*

Input: $f(x) = f_0 + f_1 x + f_2 x^2 + \cdots + f_{n-1} x^{n-1} \in \mathbb{F}_2[x]_{<n}$
Output: $g(X) = g_0 + g_1 X_1(x) + g_2 X_2(x) + \cdots + g_{n-1} X_{n-1}(x) \in \mathbb{F}_2[x]_{<n}$
Data: $y = x^{2^i} + x = s_i(x)$

if $\deg f(x) \leqslant 1$ **then**
  return $g(X) \leftarrow f_0 + f_1 X_1$;
$i \leftarrow \max(2^k)$ s.t. $\deg(s_i(x)) \leqslant \deg(f(x))$;
$h(x, y) = \hat{h}(y) \leftarrow h_0(x) + h_1(x) y + \cdots + h_{m-1}(x) y^{m-1}$; // $h_i(x) \in \mathbb{F}_2[x]$
$h'(Y) = q_0(x) + \cdots + q_{m-1} X_{(m-1)2^k} \leftarrow \mathsf{BasisCvt}(\hat{h}(y))$;
for $j = 0$ *to* $m - 1$ **do**
  $g_j(X) \leftarrow \mathsf{BasisCvt}(q_j(x))$;
return $g(X) = g_0 + g_1(X) X_{2^k} + \cdots + g_{n-1}(X) X_{(m-1)2^k}$;

Divide et impera: the Butterflies

The general idea of the LCH FFT to evaluate $f(x)$ at all points of W_k is to split f into two half sized polynomials, as well as splitting the set of points to be evaluated into W_{k-1} and $W_k \setminus W_{k-1} = \{x + v_k | x \in W_{k-1}\}$; the full algorithm is detailed in Algorithm 2.

Algorithm 2: Lin-Chung-Han FFT

Input: $f(x) = g(X) = g_0 + g_1 X_1(x) + \cdots + g_{n-1} X_{n-1}(x) \in \mathbb{F}_{2^m}[x]_{<n}$, $\alpha \in \mathbb{F}_{2^m}$

Output: $S = \{f(x) | x \in W_m\}$

1 **if** $\deg(f(x))$ **then**
2 | **return** $[g_0]$;
3 $k \leftarrow \max(i)$ s.t. $2^i \leqslant \deg(f(x))$;
4 $g(X) \leftarrow p_0(X) + X_{2^k} \cdot p_1(X)$;
5 $h_0(X) \leftarrow p_0(X) + s_k(\alpha) \cdot p_1(X)$;
6 $h_1(X) \leftarrow h_0(X) + s_k(v_k) \cdot p_1(X)$;
7 **return** $[FFT(h_0(X), \alpha), FFT(h_1(X), v_k + \alpha)]$;

The *actual* computation, performed at lines 5 and 6, is commonly called *butterfly* and is schematized in Figure 2.1. Notice that the butterfly performs two field additions but only one field multiplication, since $s_k(v_k) = 1$ in Cantor basis.

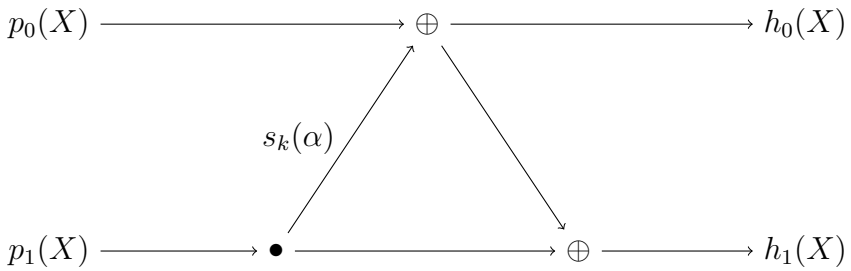


Figure 2.1: Butterfly unit

Frobenius Additive FFT

In [vDHL17] the authors showed that, by using the Frobenius map, it is possible to derive more values of polynomials from less evaluation points: given $f(x) \in \mathbb{F}_2[x]_{<n}$, it is possible to prove that

$$f(\phi_2(a)) = \phi_2(f(a)) \tag{2.9}$$

meaning that we can derive the value of f at $\phi_2(a)$ by applying the Frobenius map to $f(a)$. Our goal now is, given a polynomial $f(x) \in \mathbb{F}_2[x]_{<n}$, to design a set $\Sigma \subset \mathbb{F}_{2^m}$ for m

a power of 2 such that we can derive n values of f from Σ.

Frobenius partition Consider the Frobenius map in $\mathbb{F}_{2^m}$; with respect to the Cantor basis, it has been shown in [LCK+18] that the order of the Frobenius map is

$$Ord_{\phi_2}(v_i) = 2^{\lfloor \log_2 i \rfloor + 1} \tag{2.10}$$

meaning that the maximum order for $v_i \in \{v_{m/2}, \cdots, v_{m-1}\}$ is m. If we define $l_m = \log_2 m$, $l_n = \log_2 n$ and $l = l_n - l_m$, we can design a set with maximum order as:

$$\Sigma := v_{l+m/2} + W_l = \{v_{l+m/2} + x \mid x \in W_l\} \tag{2.11}$$

with size $n' = |\Sigma| = |W_l| = 2^l = n/m$ and order m. By applying the Frobenius map to Σ repeatedly, it is possible to prove that

$$\Omega = \Sigma \cup \phi_2(\Sigma) \cup \cdots \cup \phi_2^{m-1}(\Sigma) \tag{2.12}$$

with $|\Omega| = n$ and Σ is a Frobenius partition of Ω [LCK+18].

Polynomial evaluation Now that we built the Frobenius partition, we have to evaluate f on it: we define the linear map

$$E_\Sigma : f(x) \mapsto \{f(v_{l+m/2} + w) \mid w \in W_l\} \tag{2.13}$$

that evaluates $f(x)$ at $n' < n$ points; however, since the points are in $\mathbb{F}_{2^m}$, we get $n' \cdot m = n/m \cdot m = n$ values in $\mathbb{F}_2$.

To perform E_Σ we can use the same algorithm used for the LCH FFT, with some modifications: recalling that Algorithm 1 depends only on the polynomial and that Algorithm 2 performs the actual butterfly computations, we only have to adapt the latter to the new set of evaluation points. In particular, we pretend to evaluate f at $v_{l+m/2} + W_l$, but *truncate* the first l_m layers of the butterflies, as we do not need the whole evaluation.

> **Example:**
> Let's consider the calculation of the first value f_0 for $m = 4$ and a generic n. Starting from 4 points $(a_{0 \cdot n'}, a_{1 \cdot n'}, a_{2 \cdot n'}, a_{3 \cdot n'})$, each from $\mathbb{F}_2$, the first $l_m = 2$ layers of the butterfly stage are truncated as shown in Figure 2.2. The multiplications, denoted by $(\bullet)$, produce the output:
>
> $$f_0 = a_{0 \cdot n'} + s_{l+1}(v_{l+m/2}) \cdot a_{1 \cdot n'} + s_{l+2}(v_{l+m/2}) \cdot a_{2 \cdot n'} + s_{l+1}(v_{l+m/2}) \cdot s_{l+2}(v_{l+m/2}) \cdot a_{3 \cdot n'}$$

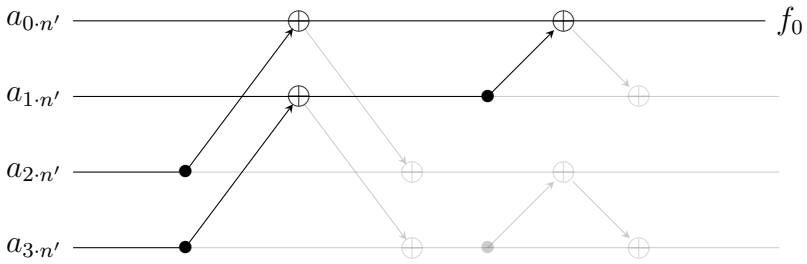


Figure 2.2: A 2-layer Truncated FFT

Because of the recursive nature of the vanishing polynomial, we can rewrite the result above as:

$$f_0 = a_{0\cdot n'} + v_{m/2-1} \cdot a_{1\cdot n'} + v_{m/2-2} \cdot a_{2\cdot n'} + v_{m/2-2} \cdot a_{3\cdot n'}$$

where the multipliers are independent of l for a fixed m. Since the multipliers are in $\mathbb{F}_{2^m}$ the output f_0 is in the same field, thus not requiring the computation of the remaining values.

After this step, we can finally carry out the remaining butterfly computations, evaluating the encoded polynomials at the points in Σ.

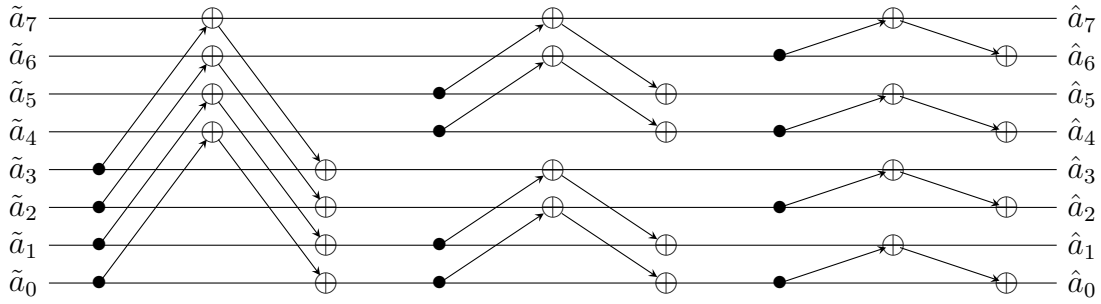


Figure 2.3: A size-8 FFT

We can finally summarize the Frobenius Additive FFT algorithm in Algorithm 3.

Algorithm 3: Frobenius Additive FFT Multiplication

Input: $A(x) = a_0 + a_1 x + \cdots + a_{n-1}x^{n-1} \in \mathbb{F}_2[x]_{<n}$,
$B(x) = b_0 + b_1 x + \cdots + b_{n-1}x^{n-1} \in \mathbb{F}_2[x]_{<n}$

Output: $C(x) = c_0 + c_1 x + \cdots + c_{n-1}x^{n-1} = A(x) \cdot B(x) \mod (x^n + 1) \in \mathbb{F}_2[x]_{<n}$

1 $\bar{A}(X) \leftarrow \text{BasisCvt}(A(x)) \in \mathbb{F}_2[x]_{<n}$;
2 $\bar{B}(X) \leftarrow \text{BasisCvt}(B(x)) \in \mathbb{F}_2[x]_{<n}$;
3 $\tilde{A}(X) \leftarrow \text{Encode}(\bar{A}(X)) \in \mathbb{F}_{2^m}[x]_{<n'}$;
4 $\tilde{B}(X) \leftarrow \text{Encode}(\bar{B}(X)) \in \mathbb{F}_{2^m}[x]_{<n'}$;
5 $[\hat{a}_1, \cdots, \hat{a}_{n'}] \leftarrow \text{Butterflies}(\tilde{A}(X), \Sigma) \in \mathbb{F}_{2^m}^{n'}$;
6 $[\hat{b}_1, \cdots, \hat{b}_{n'}] \leftarrow \text{Butterflies}(\tilde{B}(X), \Sigma) \in \mathbb{F}_{2^m}^{n'}$;
7 $\hat{C} = [\hat{c}_1, \cdots, \hat{c}_{n'}] \leftarrow [\hat{a}_1 \cdot \hat{b}_1, \cdots, \hat{a}_{n'} \cdot \hat{b}_{n'}]$;
8 $\tilde{C}(X) \leftarrow \text{Butterflies}^{-1}(\hat{C}, \Sigma) \in \mathbb{F}_{2^m}[x]_{<n'}$;
9 $\bar{C}(X) \leftarrow \text{Decode}(\tilde{C}(X)) \in \mathbb{F}_2[x]_{<n}$;
10 $C(x) \leftarrow \text{BasisCvt}^{-1}(\bar{C}(X)) \in \mathbb{F}_2[x]_{<n}$;
11 **return** $C(x)$

2.3. Coding Theory

In this section we give the basic concepts and definitions of coding theory ([MS77]), required to formally define the problems on which HQC and similar cryptosystems are based.

2.3.1. General definitions

Definition 2.3.1 (Circulant matrix)**.** Let $\mathbf{v} = (v_0, \cdots, v_{n-1}) \in \mathcal{V}$. The circulant matrix induced by $\mathbf{v}$ is defined as a matrix whose i-th row is the vector $\mathbf{v}$ shifted by i positions to the right:

$$\mathbf{circ}(\mathbf{v}) = \begin{pmatrix} v_0 & v_1 & \cdots & v_{n-1} \\ v_{n-1} & v_0 & \cdots & v_{n-2} \\ \vdots & \vdots & \ddots & \vdots \\ v_1 & v_2 & \cdots & v_0 \end{pmatrix} \in \mathbb{F}_2^{n \times n}$$

The algebra of $n \times n$ circulant matrices over $\mathbb{F}_2$ is isomorphic to the algebra of polynomials in the ring $\mathcal{R}$:

$$\mathbf{circ}(\mathbf{v}) \simeq v_0 + v_1 x + v_2 x^2 + \cdots + x_{n-1}x^{n-1}$$

Definition 2.3.2 (Linear q-ary Codes)**.** A Linear q-ary code $\mathcal{C}$ of lenght n and dimension k (denoted as $[n, k]$) is a subspace of $\mathbb{F}_q^n$.
Due to their vast use in cryptography (and in particular in this work), the following definitions and results are relative to binary codes ($q = 2$).

Definition 2.3.3 (Linear Codes)**.** A linear code $\mathcal{C}$ of lenght n and dimension k (denoted as $[n, k]$) is a subspace of $\mathcal{V}$ of dimension k; the elements of $\mathcal{C}$ are called *codewords*.
Due to the linearity of $\mathcal{V}$, any linear combination of two codewords is also a codeword.

Definition 2.3.4 (Generator Matrix)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The matrix $\mathbf{G} \in \mathbb{F}_2^{k\times n}$ is called the generator matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{mG}, \text{ for } \mathbf{m} \in \mathbb{F}_2^k\}$$

Definition 2.3.5 (Parity-Check Matrix)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k)\times n}$ is called the parity-check matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{v} \in \mathbb{F}_2^n \text{ such that } \mathbf{Hv}^\top = 0\}$$

In other words, $\mathbf{H}$ is the generator matrix of the dual code $\mathcal{C}^\perp$.

Definition 2.3.6 (Rank)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The rank of $\mathcal{C}$ is defined as the dimension of the space spanned by its codewords or, in other words, the rank of its parity-check matrix.

$$r(\mathcal{C}) = r(\mathbf{H}) = n - k$$

A high rank code can correct more errors with fewer parity-check equations, but it usually is harder to construct and decode.

Definition 2.3.7 (Syndrome)**.** Let $\mathcal{C} = [n, k]$ be a code, $\mathbf{H}$ its parity matrix and $\mathbf{v} \in \mathbb{F}_2^n$ a word. The syndrome of $\mathbf{v}$ is defined as

$$\sigma(\mathbf{v}) = \mathbf{Hv}^\top$$

From this definition, we can say that a vector $\mathbf{v}$ is a codeword if and only if its syndrome is the zero vector.

Definition 2.3.8 (Rate)**.** Let $\mathcal{C} = [n, k]$ be a code. The rate of $\mathcal{C}$ is defined as $R = \frac{k}{n}$.

The rate of a code is a measure of its efficiency: with a higher rate we can transmit more information with the same amount of bits, at the cost of reduced redundancy (and thus

reduced error correcting capability).

Definition 2.3.9 (Minimum distance)**.** Let $\mathcal{C} = [n, k]$ be a code and ω a norm on $\mathcal{V}$. The minimum distance of $\mathcal{C}$ is defined as

$$d = \min_{\mathbf{v},\mathbf{w}\in\mathcal{C},\mathbf{v}\neq\mathbf{w}} \omega(\mathbf{v} - \mathbf{w})$$

The length, dimension and minimum distance of a linear code $\mathcal{C} = [n, k, d]$ are tied by the Singleton bound [Sin64]:

$$d \leq n - k + 1$$

If the equality holds, $\mathcal{C}$ is called a maximum distance separable (MDS) code, and it has desirable properties in terms of error correction.

Definition 2.3.10 (Error Correction Capability)**.** A linear code $\mathcal{C} = [n, k, d]$ can decode arbitrary codewords with at most $\delta = \lfloor \frac{d-1}{2} \rfloor$ errors. If $\mathcal{C}$ is a MDS code, we can rewrite this as $\delta = \lfloor \frac{n-k}{2} \rfloor$.

Definition 2.3.11 (Gilbert-Varshamov bound)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The Gilbert-Varshamov bound [Gil52, Var57] states that $\mathcal{C}$ can correct at least t errors if its size satisfies

$$2^k \geq \sum_{i=0}^{t} \binom{n}{i} \tag{2.14}$$

2.3.2. Code families

Informally, when we say that a code is *good*, we mean that it satisfies some properties, such as:

- High minimum distance: the higher the minimum distance, the more errors the code can correct.
- High rate: the higher the rate, the more information we can transmit with the same amount of bits.
- Efficiency: we want the encoder and decoder to be fast and simple to implement.

In this section we will describe some of the most common code families and the properties they satisfy.

Definition 2.3.12 (Cyclic Codes)**.** Let $\mathbf{c} = (c_0, \cdots, c_{n-1}) \in \mathcal{V}$ be a vector. A linear code $\mathcal{C} = [n, k]$ is Cyclic if, for any $\mathbf{v} \in \mathcal{C}$, the vector obtained by applying a circular shift to $\mathbf{v}$

is also a codeword. codeword. If we think of $\mathbf{v}$ as a polynomial in $\mathcal{R}$, we can say that $\mathcal{C}$ is cyclic if, for any $\mathbf{v} \in \mathcal{C}$, the polynomial $X \cdot \mathbf{v}$ is also a codeword.

Definition 2.3.13 (Quasi-Cyclic Codes)**.** Let $\mathbf{c} = (\mathbf{c}_0, \cdots, \mathbf{c}_{s-1}) \in \mathbb{F}_2^{sn}$ be a vector, where we consider each $\mathbf{c}_i$ as a vector in $\mathbb{F}_2^n$. A linear code $\mathcal{C} = [sn, k]$ is Quasi-Cyclic of index s if, for any $\mathbf{c} \in \mathcal{C}$, the vector obtained by applying a circular shift to every block $\mathbf{c}_i$ simultaneously is also a codeword. QC codes are a generalization of cyclic codes.

Definition 2.3.14 (Double Circulant Codes)**.** A code $\mathcal{C}$ is double circulant if it is Quasi-Cyclic of even length and index $s = 2$.

Random codes

In a linear random code the codewords are chosen at random; specifically, we can build the parity matrix $\mathbf{H}$ of the code by randomly sampling each entry from a Bernoulli distribution.
For sufficiently large n, random linear codes reach the Gilbert-Varshamov bound; moreover, the probability of decoding error decreases exponentially as the minimum distance increases. On the other hand, the computational complexity of a decoder for a random code grows exponentially with the length of the code.

Repetition codes

Repetition codes are among the simplest codes, as they are just a repetition of the same vector k times. Repetition codes can be easily decoded by majority voting, making the decoding process very fast. However, they have a very low rate and they are not efficient in terms of error correction capability.

Concatenated codes

Concatenated codes are built by combining an *inner* code $\mathcal{C}_{in} = [n_{in}, k_{in}]$ and an *outer* code $\mathcal{C}_{out} = [n_{out}, k_{out}]$: the idea is to split the input message

- Split the input message into chunks: $\mathbf{m} = (m_1, \cdots, m_{k_{out}})$
- Encode each chunk with the outer code: $\mathbf{c} = (\mathcal{C}_{out}(m_1), \cdots, \mathcal{C}_{out}(m_{k_{out}})) = (m'_1, \cdots, m'_{n_{out}})$
- Encode the resulting codeword with the inner code: $\mathcal{C}_{in}(\mathbf{c}) = (\mathcal{C}_{in}(m'_1), \cdots, \mathcal{C}_{in}(m'_{n_{out}}))$

The resulting code, denoted as $\mathcal{C}_{out} \circ \mathcal{C}_{in}$ thus has length $n_{out} \cdot n_{in}$ and dimension $k_{out} \cdot k_{in}$.

As shown by Forney in [For65], concatenated codes can be used to achieve both a polynomial-time decoder and exponentially decreasing error probability with increased

block length.

Reed-Muller and Reed-Solomon codes

Reed-Muller codes [Ree53, Mul54] are a family of codes; by defining the message lenght as

$$k = \sum_{i=0}^{r} \binom{m}{i}$$

we denote them as $\mathsf{RM}(r, m)$, where $n = 2^m$ is the lenght of the code. Reed-Muller codes are locally testable and locally decodable, meaning that it is possible to detect and correct errors in a codeword by looking at a small subset of its coordinates.

Reed-Solomon codes [RS60] are a subclass of Reed-Muller codes and they are usually referred to as $\mathsf{RS}(n, k)$, where n and k are the lenght and dimension of the code respectively.

2.3.3. Difficult problems for cryptography

The problems we are interested in are all variants of the *decoding problem*, which requires to find the closest codeword to a given word with respect to a certain metric (the Hamming weight in our case).
In the case of linear codes we have shown that the decoding problem stays the same whether one is given the syndrome of the received vector or the vector itself; for this reason, we will only consider the *Syndrome Decoding* (SD) problem.

Definition 2.3.15 (SD Distribution)**.** Given $n, k, w \in \mathbb{N}$, the distribution $\mathsf{SD}(n, k, w)$ randomly samples a parity-check matrix $\mathbf{H} \xleftarrow{\$} \mathbb{F}_2^{(n-k)\times n}$, a vector $\mathbf{x} \xleftarrow{\$} \mathbb{F}_2^n$ such that $\mathtt{wt}(\mathbf{x}) = w$ and returns $(\mathbf{H}, \sigma(\mathbf{x}))$.

Definition 2.3.16 (SD Problem)**.** Given $(\mathbf{H}, \mathbf{y}^\top) \in \mathbb{F}_2^{(n-k)\times n} \times \mathbb{F}_2^{(n-k)}$ from the $\mathsf{SD}(n, k, w)$ distribution, the SD problem requires to find a vector $\mathbf{x} \in \mathbb{F}_2^n$ such that $\sigma(\mathbf{x}) = \mathbf{y}^\top$ and $\mathtt{wt}(\mathbf{x}) = w$.

This problem has been proven to be NP-complete in [BMVT78].

Definition 2.3.17 (Decisional SD Problem)**.** Given $(\mathbf{H}, \mathbf{y}^\top) \in \mathbb{F}_2^{(n-k)\times n} \times \mathbb{F}_2^{(n-k)}$, the DSD problem requires to determine whether $(\mathbf{H}, \mathbf{y}^\top)$ comes from the SD distribution or from the random distribution over $\mathbb{F}_2^{(n-k)\times n} \times \mathbb{F}_2^{(n-k)}$.

2.4. Side Channel Attacks

Many cryptographic algorithms are considered unbreakable, regardless of the amount of computing power available; the theoretical security of such algorithms however does not always translate to real-world secure implementations.
Regardless of the means of the implementation, the electronic devices that run these cryptosystems often *leak* information about the internal state of the device, potentially exposing secrets. But how does a device leak information?

A *side channel* is some observable aspect of a cryptosystem that reveals information held within that system [Sta10]; example of side channels can be the execution time of an algorithm, the power consumption of a device or the electromagnetic radiation emitted while running some operations.
In this chapter we will describe the most common side channel attacks, how they can be used to retrieve secret information and how to mitigate them.

2.4.1. Timing attacks

In a timing attack, the attacker tries to infer information about a secret by measuring the time it takes to execute some operation: the idea behind this attack is that the time it takes to execute an operation depends on the input of the operation, and thus by measuring the execution time of the operation we can infer information about the input.

Example:

```
1   int check_pin(int *pin) {
2       int correct_pin[4] = {3, 1, 4, 2};
3
4       for (int i = 0; i < 4; i++) {
5           if (pin[i] != correct_pin[i]) {
6               return 0;
7           }
8       }
9       return 1;
10  }
11
```

Listing 2.1: A non constant-time password checker

We can think this function as a part of a bigger system, where the user enters a PIN and the system checks if it is correct. The function simply checks if the PIN entered by the user is equal to the correct PIN, and returns 1 if it is correct and 0 otherwise.

> Being the PIN composed of decimal digits, the total amount of combinations is 10^4, so on average we would need 5000 guesses to find the correct PIN; however, we can reduce the number of guesses needed to find the correct PIN by using a timing attack.
>
> We can observe that the function returns as soon as a digit of the PIN is incorrect: under the reasonable assumption that we can measure the execution time of the function, we can guess the correct PIN one digit at a time. Starting from 0000, we cycle through all the possibilities for the first digit $(0000, 1000, \cdots, 9000)$ and measure the execution time: out of the 10 measurements, the one with the highest execution time corresponds to the correct digit; we can then repeat the same process for the remaining digits, so that after at most 40 guesses we find the correct pin.

Even though this attack strategy may seem trivial and easy to mitigate, it can still wreck havoc: real-life examples of timing attacks include, for example, Spectre [KHF+19] and Meltdown [LSG+18], which affected almost all modern processors and allowed an attacker to read the memory of any process running on the same machine.

2.4.2. Power attacks

In order to carry out operations, digital devices require to be powered: the power consumption of a device is usually not fixed, as it can fluctuate based on a variety of factors. The vast majority of digital devices employs transistors, which act as switches that can be turned on or off; the input to a transistor can be thought of as a small capacitor and, as such, follows its characteristic equation:

$$I = C \cdot \frac{dV}{dt}$$

where I is the current, C is the capacitance of the capacitor and V is the voltage across the capacitor. To turn on a transistor (*i.e.* to increase the voltage across the capacitor) we need to apply a *charge* to it: because of this behavior, if we have a changing voltage on the capacitor, for example when transitioning from 0 to 1, we can observe the flow of current through the capacitor.
All operations in a digital circuit are performed by switching millions of transistors on and off: this means that the power consumption of a digital device is affected by the operations it performs, and thus can be used to infer information about the internal state of the device.

Example: RSA Square and Multiply

The Rivest-Shamir-Adleman (RSA) cryptosystem is one of the most known public-key cryptosystem. In the encryption phase, RSA produces the ciphertext c by raising the plaintext m to the power of the private key e modulo n, as shown in the following listing:

```
1   int square_and_multiply(int ptx, int exponent, int modulus) {
2       int result = 1;
3       while (exponent > 0) {
4           if (exponent % 2 == 1) {
5               result = (result * ptx) % modulus;
6           }
7           exponent = exponent / 2;
8           ptx = (ptx * ptx) % modulus;
9       }
10      return result;
11  }
12
```

Listing 2.2: RSA square and multiply

The if statement at line 4 determines the execution flow based on the bits of the secret exponent: if the current bit is 1 the result is multiplied by the plaintext (*multiply*), otherwise it is not, and we only *square* the plaintext.

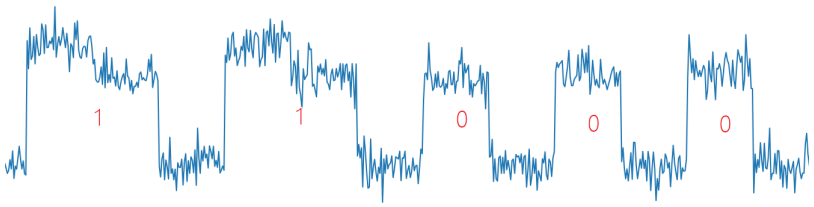


Figure 2.4: Power consumption of the RSA Square and Multiply algorithm

As shown in the figure above, by looking at the power consumption we can clearly distinguish whether the exponent bit is 1 or 0, and thus we can infer the value of the secret exponent.

Measurements and attacks

When it comes to actually measuring the power consumption of a device we have two main approaches:

- Measure the voltage across a *shunt resistor*: a resistor with a fixed value is placed between the power source and the device
- Measure the emitted electromagnetic radiation: the current flowing through the device induces magnetic field in the wires, that we can pick up with a near-field probe

Both methods have their pros and cons: the first one is more accurate, but requires to phisically install the shunt resistor on the device; the second one is non-intrusive, but the measurements are tipically noisy due to external interferences.

Regardless of the source of a power trace, there are several possibilities to carry out an attack, such as:

- Simple Power Analysis: observe the power consumption over a single trace
- Differential Power Analysis: analyze multiple executions over the same algorithm with different inputs to detect small input-dependent differences
- Template attacks: collect a large number of traces and use them to build a *template* of the power consumption of the device to detect when the device is performing a specific operation

2.4.3. Countermeasures

The existence of a side-channel does not necessarily mean that an attack is possible: in fact, there are a number of countermeasures that can mitigate the risk of a successful attack.

Regarding timing attacks, the most common countermeasure is to use *constant-time* algorithms, that take the same amount of time to execute regardless of the input; this can be achieved, for example, by replacing branching statements that depend on the inputs.

A very effective way to avoid leakage, especially with respect to power attacks, is to randomize the inputs of the algorithm without affecting the output. To do so, we can *mask* the inputs: the basic principle of masking [PR13] is to split every sensitive value x

into $d+1$ *shares*, in such a way that the following relation holds:

$$x = x_0 \odot x_1 \odot \cdots \odot x_d \tag{2.15}$$

The shares $x_1, \cdots, x_d$ are called *masks* and are randomly sampled, while the secret x_0 is processed in such a way that the relation above holds. The goal of masking is to make the sensitive value indistinguishable from a random value, so that the observation of a side channel does not reveal any sensitive information, but only random noise. There is no particular constraint on the choice of the operation $\odot$ as long as it preserves the relation above; in practice, the most common choice is to use the XOR operation, as it is very fast and easy to implement: such a masking scheme (e.g. [ISW+03]) is often referred to as *boolean* masking. As proven in [CJRR99], a masking scheme with d shares is a sound counermeasure against a side-channel attack of d-th order, where an attacker can have up to d *probes* on the target system; this means that a $d+1$-th order attack could be possible, but such attacks become impractical as d increases.

2.5. Hamming Quasi-Cyclic

Hamming Quasi-Cyclic [MAB+18] is a fourth round candidate in the NIST Post-Quantum cryptography standardization process. HQC is a code-based cryptosystem, that includes both a public-key encryption (HQC.PKE) and a key encapsulation mechanism (HQC.KEM). In this section we will describe the inner workings of HQC and show its theoretical security; moreover, we will give an overview of the currently known attacks against HQC and their practical impact.

2.5.1. Encryption scheme

We can think of a generic encryption scheme as a tuple of four polynomial time algorithms:

- Setup(1^λ): generates the parameters param based on the security parameter λ.
- Keygen(param): generates a public key pk and a secret key sk based on the global parameters param.
- Encrypt(pk, $\mathbf{m}$, θ): outputs the ciphertext $\mathbf{c}$ for the message $\mathbf{m}$ with the randomness θ under the public key pk.
- Decrypt(sk, $\mathbf{c}$): outputs the message $\mathbf{m}$ for the ciphertext $\mathbf{c}$ (or $\perp$ if the ciphertext is invalid).

2.5.2. Public Key Encryption

HQC uses two different codes:

- A publicly available decodable code $\mathcal{C} = [n, k]$ generated by $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ with an efficient decoding algorithm $\mathcal{C}.\mathsf{Decode}()$ which can correct up to δ errors.
- A random double-circulant $[2n, n]$ code with parity-check matrix $\mathbf{H} = (\mathbf{I_n}, \mathbf{circ}(\mathbf{h}))$.

The HQC.PKE defines the four algorithms of an encryption scheme as follows:

- $\mathsf{Setup}(1^\lambda)$: generates the set of parameters $\mathsf{param} = (n, k, \delta, \omega, \omega_r, \omega_e)$
- $\mathsf{Keygen}(\mathsf{param})$: samples $\mathbf{h} \xleftarrow{\$} \mathcal{R}$ and $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathcal{R} \times \mathcal{R}$ such that $\mathtt{wt}(\mathbf{x}) = \mathtt{wt}(\mathbf{y}) = \omega$; the keypair is defined as $\mathsf{sk} = (\mathbf{x}, \mathbf{y})$ and $\mathsf{pk} = (\mathbf{h}, \mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y})$.
- $\mathsf{Encrypt}(\mathsf{pk}, \mathbf{m}, \theta)$: samples $\mathbf{e} \xleftarrow{\$} \mathcal{R}$, $(\mathbf{r_1}, \mathbf{r_2}) \xleftarrow{\$} \mathcal{R} \times \mathcal{R}$ such that $\mathtt{wt}(\mathbf{e}) = \omega_e$ and $\mathtt{wt}(\mathbf{r_1}) = \mathtt{wt}(\mathbf{r_2}) = \omega_e$; the ciphertext is defined as the pair $(\mathbf{u}, \mathbf{v})$, where $\mathbf{u} = \mathbf{r_1} + \mathbf{h} \cdot \mathbf{r_2}$ and $\mathbf{v} = \mathbf{mG} + \mathbf{s} \cdot \mathbf{r_2} + \mathbf{e}$.
- $\mathsf{Decrypt}(\mathsf{sk}, \mathbf{c})$: outputs $\mathcal{C}.\mathsf{Decode}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$.

The idea at the core of HQC.PKE is to encode the message ($\mathbf{mG}$) and then add random errors that can be removed only by knowing the secret key: the code $\mathcal{C}$ correctly decodes $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$ whenever

$$\begin{aligned} \mathtt{wt}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y}) &= \mathtt{wt}(\mathbf{s} \cdot \mathbf{r_2} - \mathbf{u} \cdot \mathbf{y} + \mathbf{e}) \\ &= \mathtt{wt}((\mathbf{x} + \mathbf{h} \cdot \mathbf{y}) \cdot \mathbf{r_2} - (\mathbf{r_1} + \mathbf{h} \cdot \mathbf{r_2}) \cdot \mathbf{y} + \mathbf{e}) \\ &= \mathtt{wt}(\mathbf{x} \cdot \mathbf{r_2} - \mathbf{r_1} \cdot \mathbf{y} + \mathbf{e}) \leqslant \delta \end{aligned}$$

The PKE scheme has been proven to be IND-CPA secure in [AMBD+18].

2.5.3. Key Encapsulation Mechanism

In practice, public key encryption is not used to encrypt large amounts of data because of its high exectution time compared to symmetric encryption. For this reason, we use public key encryption to exchange a shared secret key, which is then used to encrypt the actual data: the practice of using a public key encryption scheme to exchange a shared secret key is referred to as key encapsulation mechanism (KEM).
The Fujisaki-Okamoto transformation [FO13] is a technique that allows to transform a *weak* IND-CPA encryption scheme into a *stronger* IND-CCA2 key encapsulation mechanism.

Let $\mathcal{E}$ an instance of HQC.PKE and $\mathcal{G}, \mathcal{H}$ and $\mathcal{K}$ hash functions. The HQC.KEM scheme is defined as follows:

- $\mathsf{Setup}(1^\lambda)$: generates the set of parameters as in HQC.PKE.Setup, where k will be the key size.
- $\mathsf{Keygen}(\mathsf{param})$: generates a keypair as in HQC.PKE.Keygen.
- $\mathsf{Encapsulate}(\mathsf{pk})$: sample $\mathbf{m} \xleftarrow{\$} \mathbb{F}_2^k$ to derive the randomness $\theta = \mathcal{G}(\mathbf{m})$. Generate the ciphertext as $\mathbf{c} = (\mathbf{u}, \mathbf{v}) \leftarrow \mathcal{E}.\mathsf{Encrypt}(\mathsf{pk}, \mathbf{m}, \theta)$ and derive the symmetric key as $K \leftarrow \mathcal{K}(\mathbf{m}, \mathbf{c})$; send $(\mathbf{c}, \mathbf{d} = \mathcal{H}(\mathbf{m}))$.
- $\mathsf{Decapsulate}(\mathsf{sk},\ \mathbf{c}, \mathbf{d})$: decrypt the ciphertext as $\mathbf{m}' = \mathcal{E}.\mathsf{Decrypt}(\mathsf{sk}, \mathbf{c})$, compute $\theta' = \mathcal{G}(\mathbf{m}')$ and re-encrypt to get $\mathbf{c}' \leftarrow \mathcal{E}.\mathsf{Encrypt}(\mathsf{pk}, \mathbf{m}', \theta)$. If $\mathbf{c} \neq \mathbf{c}'$ or $\mathbf{d} \neq \mathcal{H}(\mathbf{m}')$ return $\perp$, otherwise derive the shared key $\mathcal{K}(\mathbf{m}', \mathbf{c}')$.

It is important in this setting to pick different hash functions for $\mathcal{H}$ and $\mathcal{G}$: if not, we would have that $\theta = \mathcal{G}(\mathbf{m}) = \mathcal{H}(\mathbf{m}) = \mathbf{d}$, thus publishing the randomness used to generate $\mathbf{r}_1, \mathbf{r}_2$ and $\mathbf{e}$. In such a scenario, an eavesdropper could retrieve the message $\mathbf{m}$ from $\mathbf{mG} + \mathbf{sr_2} + \mathbf{e}$, which is the seed for the secret shared key.

2.5.4. Parameters

HQC has three different parameters sets, for category 1, 3 and 5 respectively. We will refer to the implementations of these sets as `hqc-128`, `hqc-192` and `hqc-256`.

Starting from Round 3, the public code with generator matrix $\mathbf{G}$ is instantiated with a concatenated code, using Reed-Solomon and Reed-Muller codes as outer and inner codes, with n_1 and n_2 lenght of the respective codes; the length of the message is n, which is the smallest prime greater than $n_1 n_2$ such that $ord_n(2) = n - 1$. The random code with parity-check matrix $\mathbf{H}$ is instantiated by randomly sampling the vector $\mathbf{h}$ from $\mathbb{F}_2^n$. The parameters for the three sets are chosen to bound the decryption failure rate (DFR), and are summarized in Table 2.1.

Table 2.1: Parameters of HQC

Instance	n_1	n_2	n	ω	$\omega_r = \omega_e$	DFR
`hqc-128`	46	384	17'669	66	75	$< 2^{-128}$
`hqc-192`	56	640	35'851	100	114	$< 2^{-192}$
`hqc-256`	90	640	57'637	131	149	$< 2^{-256}$

3 | Implementation

At the time of writing there are no publicly available implementations of HQC specifically targeting microcontrollers. In this section we describe our implementation of HQC targeting ARM Cortex-M4 microcontrollers, showing the implementative details for various multiplication algorithms; moreover, we show the first fully masked implementation of HQC, as a countermeasure against power side-channel attacks.

3.1. Polynomial multiplication

From a computational standpoint, the most expensive operations in HQC are by far the polynomial multiplication. In this section we will describe the various methods employed to implement polynomial multiplication in HQC.

The polynomials $v \in \mathcal{R}$ can be represented in two possible forms:

- sparse, as a vector of size ω coefficients, $\omega = \texttt{wt}(v)$, where each entry corresponds to the degree of a non-zero coefficient.
- dense, as a vector of size $\lceil \frac{n}{\texttt{ws}} \rceil$, where `ws` is the word size of the target architecture.

With the aforementioned parameters, the objects on which HQC operates require the amounts of memory summarized in Table 3.1.

Table 3.1: Memory consumption (in bytes)

Instance	Message	Secret Key	Public Key	Ciphertext	Dense Poly	Sparse Poly
`hqc-128`	16	2289	2249	4481	2209	300
`hqc-192`	24	4562	4522	9026	4482	456
`hqc-256`	32	7285	7245	14469	7205	596

Note that the secret key size refers to the memory required to store a *processable* version of if; the secret key can also be stored as a 40B seed, which is then used to generate the actual key material.

Due to its large memory footprint and the limited resources of the target platform, we will consider only the `hqc-128` instance.

3.1.1. Shift and Add

In the third round reference implementation of HQC, the polynomial multiplication is implemented using a variant of the schoolbook *shift and add* method, recalling that the multiplication in $\mathcal{R}$ of a polynomial by a monomial of degree i can be implemented as a circular shift to the left ($\lll_n$) of i positions. Given $a(x)$ in sparse form and $b(x)$ in dense form, the algorithms multiplies $b(x)$ by x^w, for all ws, where w is a non-zero coefficient of $a(x)$, and then adds the results together:

Algorithm 4: Sparse polynomial multiplication

Input: $a(x) \in \mathcal{R}_\omega$ in sparse form, $b(x) \in \mathcal{R}$ in dense form

Output: $c(x) = a(x) \cdot b(x) \in \mathcal{R}$ in dense form

1 **for** $i \leftarrow 0$ **to** ts **do**
2 $\quad \texttt{table}[i] \leftarrow b(x) \lll_n i$;
3 **for** $i \leftarrow 0$ **to** $wt(a(x))$ **do**
4 $\quad w \leftarrow a(x)[i]$;
5 $\quad \texttt{dec} \leftarrow w \mod \texttt{ts}$;
6 $\quad \texttt{s} \leftarrow \lfloor \frac{w}{\texttt{ts}} \rfloor$;
7 $\quad \texttt{tmp} \leftarrow x^{\texttt{s}} \cdot x^{\texttt{ts}}$;
8 $\quad c(x) \leftarrow c(x) + \texttt{table}[\texttt{dec}] \cdot \texttt{tmp}$;
9 **return** $c(x) \mod (x^n + 1)$;

The idea of this variant is to precompute the first `ts` shifts of $b(x)$ and store them in a table, where `ts` is set to 16; this allows to slightly improve the performance of the algorithm and to shuffle the memory accesses to `table`.
Since a is very sparse, this approach is extremely fast; however, the high speed achieved is counterbalanced by the non-constant time execution of the algorithm, which is a major drawback for the security of the implementation.

3.1.2. Constant-time shift and add

Due to the high sparsity of one of the operands, we would like to exploit the fact that the algorithm is very fast to implement a constant-time version of it; such an algorithm is implemented in LEDAcrypt [BBC+20] and is based on *shifts with conditional commitment*.

Algorithm 5: Constant-time monomial multiplication

Input: a exponent of a monomial $x^a \in \mathbb{Z}_2[x]$ with $0 \leqslant a < n$,
$b(x) \in \mathcal{R}$ in dense form

Output: $c(x) = x^a \cdot b(x) \in \mathcal{R}$ in dense form

Data: $m = \lceil \log_2(a) \rceil$ number of bits needed to encode a in binary,
i.e. $a = (\beta_{m-1}, \cdots, \beta_0)_{\texttt{bin}}$

1 $c(x) \leftarrow b(x) \lll_n (a \mod \texttt{ws})$;
2 **for** $i \leftarrow \log_2(ws)$ **to** $(m-1)$ **do**
3 $\quad \texttt{tmp} \leftarrow b(x) \lll_n i$;
4 $\quad c(x) \leftarrow c(x) + \texttt{tmp}$;
5 $\quad \texttt{cond} \leftarrow (\beta_i == 1)$;
6 $\quad \texttt{mask} \leftarrow 0 - (\texttt{WORD})(\texttt{cond})$;
7 $\quad c(x) \leftarrow c(x) + ((c(x) \wedge \texttt{mask}) \vee (\texttt{tmp}(x) \wedge \texttt{mask}))$;
8 **return** $c(x)$;

With this algorith we compute the i-th shift of $b(x)$ regardless of the value of β_i, but we sum it to the result only if $\beta_i = 1$ (the mask at line 6 contains either all zeros or all ones, '$\vee$' and '$\wedge$' correspond to the bitwise OR and AND respectively).

3.1.3. Frobenius Additive FFT

The implementation of the FAFFT is based on the work of Chen *et.al.* [CCK21]; for the sake of clarity, we highlight the core ideas of this implementation.

As shown in Algorithm 3, the FAFFT multiplication is composed of four components (with the relative inverses): basis conversion, encoding, butterflies and pointwise multiplication. Our implementation targets a 32-bit platform, hence we pick $m = 32$ for the ease of representation of the operands in memory; recalling Equation 2.11, the optimal Frobenius partition for our parameters is $\Sigma = v_{27} + W_{11}$. Since the degree of the operands is fixed, it is possible to optimize these algorithms by unrolling the recursive calls and precomputing the necessary tables.

Pointwise multiplication

To carry out the 2048 pointwise multiplications in $\mathbb{F}_{2^{32}}$ the field elements are represented in a bitsliced format, where each element is stored in a memory word. Given a and b elements in $\mathbb{F}_{2^{32}}$, we can compute their product by splitting them as $a_0 + a_1 \cdot x^{16}$ and $b_0 + b_1 \cdot x^{16}$, and computing the product using the Karatsuba multiplication algorithm in $\mathbb{F}_{2^{16}}$.

Basis conversion

In Algorithm 1, we show how to convert a polynomial from the canonical to the *novelpoly* basis by recursively dividing the input by the vanishing polynomials $s_i(x)$. Since the division by the vanishing polynomial (of largest degree possible) always produces only $h_0(x)$ and $h_1(x)$, we can unroll the recursive calls and transform the algorithm in a binary circuit with only `XOR` operations; moreover, we can remove all the operations on coefficients of the input polynomial that we know are zero.

Example:
Each step splits the input into quotient and remainder, so we can represent the whole algorithm as a tree, where a node's left child is the remainder and the right child is the quotient: by doing so, at the last step of the recursion we can read the leaves, from left to right, as the coefficients $g_0, g_1, \cdots$ of the polynomial in the novelpoly basis.

Let's take $f(x) = x^6$ in monomial basis, and let's apply the basis conversion algorithm to it. The greatest i such that $2^{2^i} \leqslant \deg(f(x))$ is $i = 2$, so at the various levels of the tree we will divide the inputs by $s_2(x) = x^4 + x$ and $s_1(x) = x^2 + x$:

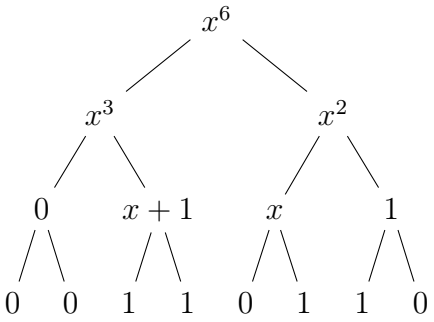


Figure 3.1: Expansion of the basis conversion algorithm for $f(x) = x^6$

hence the representation of x^6 in novelpoly basis is $X_2(x) + X_3(x) + X_5(x) + X_6(x)$.

If we repeat the same procedure with all the required monomials (of degree less than 8 for this case) we can obtain the following circuit that maps coefficients of the canonical basis f_i to coefficients of the novelpoly basis g_i, where the dashed line marks the separation between the first two levels of the tree after the root:

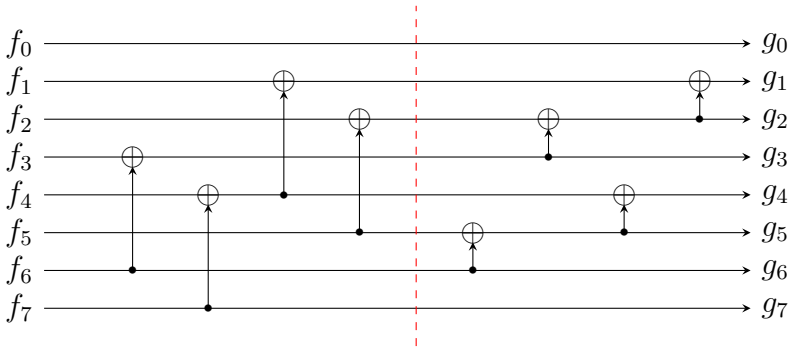


Figure 3.2: Basis conversion, $n < 8$

Encoding

As shown previously, in the encoding step the multiplication in the truncated butterflies have as one of the factors the basis vectors $(s_{l+1}(v_{l+m/2}), \cdots, s_{l+l_m}(v_{l+m/2})) = (v_{m/2-1}, \cdots, v_{m/2-l_m})$; for a fixed m these vectors are always the same, so we can rewrite this step as a vector-matrix product, summarized in Algorithm 6.

Algorithm 6: Encoding, as a vector-matrix product

Input: $f(X) = f_0 + f_1X_1 + \cdots + f_{n-1}X_{n-1} \in \mathbb{F}_2[x]_{<n}$

Output: $g(X) = g_0 + g_1X_1 + \cdots + g_{n'-1}X_{n'-1} \in \mathbb{F}_{2^m}[x]_{<n'}$

1 **for** $i \leftarrow 0$ **to** $n' - 1$ **do**

2 $\quad r_i \leftarrow \prod_{j=0}^{l_m-1}(v_{m/2-j-1})^{j_i}$

3 **for** $i \leftarrow 0$ **to** $n' - 1$ **do**

4 $\quad g_i \leftarrow \sum_{j=0}^{m-1} f_{j \cdot n'+i} \cdot r_i$

5 **return** $g(X) = g_0 + g_1X_1 + \cdots + g_{n'-1}X_{n'-1}$

Since the matrix coefficient is fixed the encoding step is a linear map, hence we can generate a binary circuit to carry out this step.

Butterflies

The multiplication step in the i-th layer of the butterflies is always of the form $\beta \cdot p_1(s_1(\beta))$, with $\beta \in v_{l+m/2-i} + W_{l-i}$, so it is possible to precompute and store the values of β in a static table. Moreover, by noticing that $\beta = v + w$ with $w \in W_{l-i}$, we can rewrite a generic multiplication $\beta \cdot \gamma$ as $\gamma \cdot v + \gamma \cdot w$: the multiplication by v is performed as a linear map in the same fashion as the encoding step, while the multiplication by w is performed by carrying out $32/2^{\lceil \log_2(l-i) \rceil}$ multiplications in the subfield of size $2^{2^{\lceil \log_2(l-i) \rceil}}$, as w is an element of such subfield. This approach, as shown empirically in [CCK21], takes the least amount of cycles among the possible approaches.

3.2. Masking

Unlike many other cryptosystems, HQC is *linear* in all its operations, making its primitives to be easily masked entirely. In the next algorithms we will use the following notation and conventions:

- d: the amount of shares
- $[\![a]\!] = (a_0, \ldots, a_{d-1})$: additive share splitting of a in d shares over $\mathcal{R}$
- $\xleftarrow{\$}$: uniform sampling from the right hand side of the equation
- $(a + \sum_{i=0}^{d-2} \texttt{rnd}_i)$: masking of a with $d-1$ masks sampled from the same field of a
- COLLAPSE(a, $[\![b]\!]$): sum all shares of b, in order, to a

In our implementation we will consider only the case $d = 2$.

3.2.1. Keygen

The masking of the key generation algorithm is straightforward: after sampling the private key, we mask y and multiply by h: the multiplication between a polynomial and a share-split is performed by multiplying each share by h, so the result is itself share-split.

Algorithm 7: Keygen, masked version

Input: none

Output: $\mathcal{K}_{pub} \in \mathcal{R} \times \mathcal{R}; \mathcal{K}_{pri} \in \mathcal{R} \times \mathcal{R}$

1 $h \xleftarrow{\$} \mathcal{R}$;
2 $(x, y) \xleftarrow{\$} \mathcal{R}_\omega \times \mathcal{R}_\omega$;
3 $[\![y]\!] \leftarrow (y + \sum_{i=0}^{d-2} \texttt{rnd}_i)$;
4 $[\![s]\!] \leftarrow h[\![y]\!]$;
5 $[\![s]\!] \leftarrow x + [\![s]\!]$; `// sum` x `only to the first share`
6 **return** $\mathcal{K}_{pri} = (x, y)$, $\mathcal{K}_{pub} = (h, \text{COLLAPSE}([\![s]\!]))$;

3.2.2. Encrypt

In the encryption it is important to protect, other than the message m and the public key s, also the random polynomials e, r_1 and r_2, as their leakage would allow an attacker to remove the *excess* errors and decode m from the ciphertext.

Algorithm 8: Encrypt($\mathcal{K}_{pub}$, m), masked version

Data: $\mathbf{G}$: a $\mathbb{F}_2^{k\times n}$ matrix generating an efficiently correctable code
Input: $\mathcal{K}_{pub}$, $m \in \{0,1\}^k$
Output: $c = (u, v) \in \mathcal{R} \times \mathcal{R}$

1 $e \xleftarrow{\$} \mathcal{R}_{\omega_e}$;
2 $(r_1, r_2) \xleftarrow{\$} \mathcal{R}_{\omega_r} \times \mathcal{R}_{\omega_r}$;
3 $[\![r_1]\!] \leftarrow (r_1 + \sum_{i=0}^{d-2} \texttt{rnd}_i)$;
4 $[\![r_2]\!] \leftarrow (r_2 + \sum_{i=0}^{d-2} \texttt{rnd}_i)$;
5 $[\![u]\!] \leftarrow [\![r_1]\!] + h[\![r_2]\!]$;
6 $[\![m]\!] \leftarrow (m + \sum_{i=0}^{d-2} \texttt{rnd}_i)$;
7 $[\![v]\!] \leftarrow [\![m]\!]\mathbf{G} + s[\![r_2]\!] + e$;
8 **return** (Collapse($[\![u]\!]$), Collapse($[\![v]\!]$));

When calculating $[\![v]\!]$ it is important to perform the operations reported at line 7 from left to right, to avoid leaking information about e; in alternative, we can also mask e and then sum it to $[\![v]\!]$.

3.2.3. Decrypt

In the decryption we exploit the linearity of the correctable code: after masking y and multiplying by u, we generate a random codeword $[\![\bar{m}]\!]$ and encode it into $[\![\bar{c}]\!]$ to protect $v - uy$ during the decoding. After the decoding we can collapse $[\![\bar{m}]\!]$ and the decoded message: since the code is linear, the final result is the original message, as if we had performed the decoding on $v - uy$ directly.

Algorithm 9: Decrypt($\mathcal{K}_{pri}$, c), masked version

Data: $\mathbf{G}$: a $\mathbb{F}_2^{k\times n}$ matrix generating an efficiently correctable code
Input: $\mathcal{K}_{pri} = (x, y) \in \mathcal{R}_\omega \times \mathcal{R}_\omega$, $c = (u, v) \in \mathcal{R} \times \mathcal{R}$
Output: $m \in \{0,1\}^k$

1 $[\![y]\!] \leftarrow (y + \sum_{i=0}^{n-2} \texttt{rnd}_i)$;
2 $[\![uy]\!] \leftarrow u[\![y]\!]$;
3 $[\![\bar{m}]\!] \xleftarrow{\$} (\{0,1\}^k)^d$;
4 $[\![\bar{c}]\!] \leftarrow [\![\bar{m}]\!]\mathbf{G}$;
5 $a \leftarrow$ Collapse($v, [\![\bar{c}]\!]$);
6 $m \leftarrow$ Decode(Collapse($a, [\![uy]\!]$));
7 **return** Collapse($m, [\![\bar{m}]\!]$);

4 | Empirical Evaluations

We chose the STM32L476RG Nucleo board as our benchmarking platform, featuring 96KB of RAM, 1MB of Flash memory, a 32-bit ARM Cortex M4 which can be clocked up to 80MHz and a true random number generator. Due to its large memory footprint and the limited amount of memory available on our target platform, we take into consideration only the `hqc-128` instance. We use `libopencm3` (1.0.0) as firmware library and compile the project with `gcc-arm-none-eabi` (7.2.1) using the `-O3` optimization flag. The board is clocked at a range of frequencies between 16 and 80 MHz to evaluate the impact of the memory wait states, as shown in Table 4.1.

Frequency (MHz)	16	32	48	64	80
Wait States	0	1	2	3	4

Table 4.1: Wait states and corresponding frequencies

For completeness, we run the same experiments on a x86 machine, equipped with an AMD Ryzen 25000U, 8GB of DDR4 DRAM and running ArchLinux 6.2; the executables are compiled with `gcc` (12.2.1) using the `-O3` optimization flag and, for AVX implementations, the `-march=native` flag.

Welch's t-test

To evaluate the existence of an eventual side-channel vulnerability we use the Test Vector Leakage Assessment (TVLA) [SM15], based on Welch's t-test. The aim of the test is to provide a quantitative measure of how much the means of two populations differ; the t-test statistic is computed as follows:

$$t = \frac{\mu_0 - \mu_1}{\sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}} \tag{4.1}$$

where μ, s^2 and n are the mean, variance and number of samples of the two populations, respectively. Small t values give evidence to accept the null hypothesis that the two populations have the same mean; usually, for the sake of simplicity, a threshold of $|t| > 4.5$ is

defined to reject the null hypothesis with a confidence level of $99,999\%$.

4.1. Execution time

To measure the execution times on ARM we use the Data Watchpoint and Trace Cycle Count Register (`DWT_CYCCNT`), a 32-bit register that counts the number of clock cycles elapsed since its last reset with a resolution of one clock cycle; to avoid overflows, we reset the register after all the measurements of a single round. As for the x86 machine, we use the `rdtsc` instruction, that returns the current value of the processor's time stamp counter; moreover, to get accurate measurements, we lock the frequency of all cores to 2.0GHz using `cpupower` and disable the Turbo Boost feature.
In both cases, in order to reduce uncertainty, we run the experiments 1000 times and average the results.

As already mentioned, the multiplication operation is the most time-consuming operation for all primitives, hence the choice of the multiplication algorithm is crucial to achieve good performance. The execution times of the various multiplication algorithms are shown in Figure 4.1 and in Table A.1.

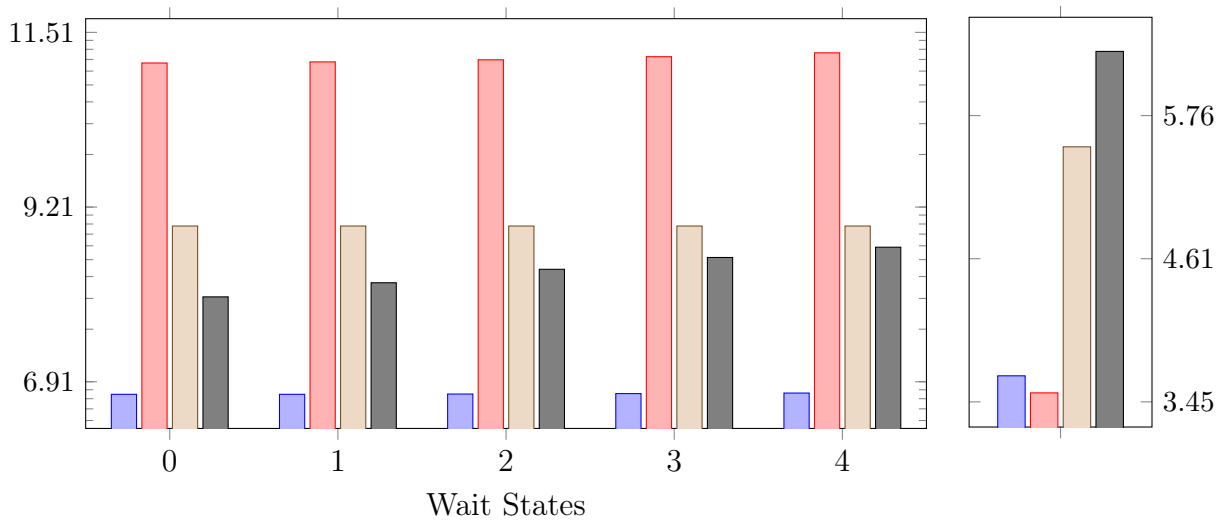


Figure 4.1: Multiplication execution times on ARM (left) and x86 (right)

As expected, the fastest multiplication algorithm on ARM (after the shift and add) is the one using the Frobenius FFT, followed by the constant-time version of shift and add. In the graph we can also see how the execution time for the Frobenius FFT is the only one that is significantly impacted by the amount of memory wait states: as explained

in Section 3.1.3 this algorithm heavily relies on precomputed tables, requiring a higher amount of memory accesses and requiring up to 92% more clock cycles; finally, we can see that the Karatsuba algorithm takes the longest amount of time. Things change dramatically on x86: on this architecture we can take advantage of AVX instructions, which allow us to carry out operations with 256-bit vectors in a single clock cycle: this benefits the Karatsuba algorithm, which is the fastest one by a large margin, while the Frobenius FFT becomes the slowest one.

These results are clearly reflected in the execution times of both the PKE and KEM schemes, as shown in Figures 4.2, 4.3. and in Tables A.2, A.3.

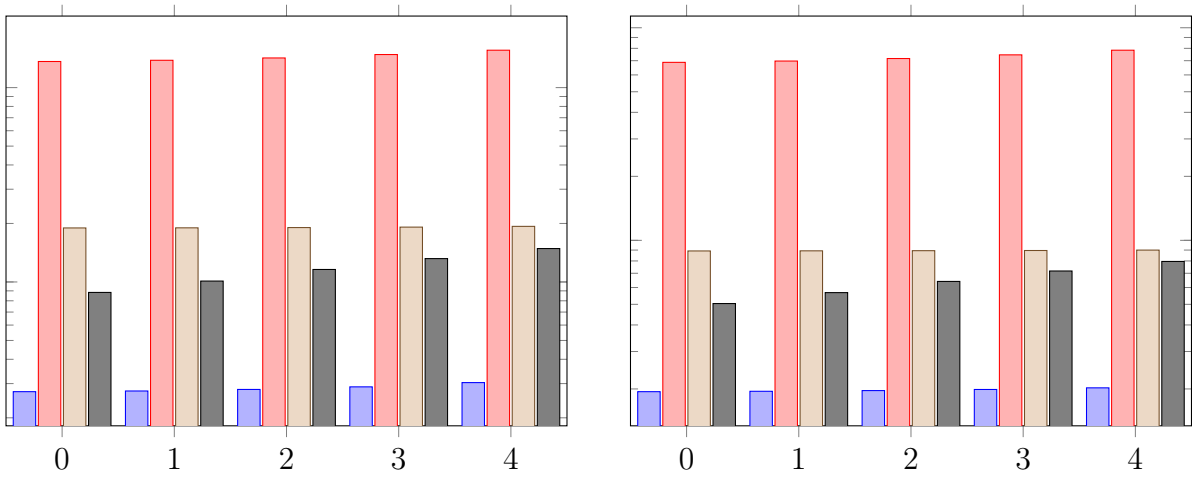


Figure 4.2: Encryption/decryption execution times on ARM

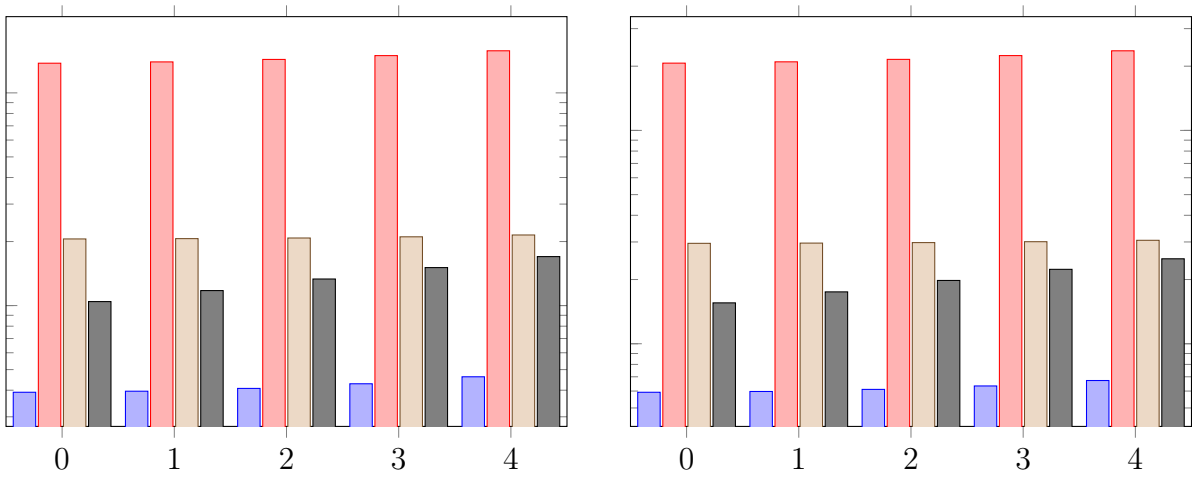


Figure 4.3: Key encapsulation/decapsulation execution times on ARM

4.2. Power measurements

Our power measuring setup employs the PicoScope 5244D oscilloscope sampling at 500MS/s, two Agilent INA-10386 amplifiers in a cascade configuration achieving a 52dB gain and a H-probe, placed on top of the STM32L476RG package. The EM traces are recorded and analyzed on a computer equipped with an AMD A8-6600K CPU, 16GB of DDR3 DRAM running Debian 10. To ensure that the encryption/decryption process runs correctly, we generate the inputs on the same machine, send them to the target via the USART1 interface and compare the outputs with the expected ones; the executables to generate the inputs and verify the outputs are compiled with `gcc` (9.3.0) on the same machine, using the `-O3` optimization flag.

5 | Conclusions

Bibliography

[AMBD+18] Carlos Aguilar-Melchor, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, and Gilles Zémor. Efficient encryption from random quasi-cyclic codes. *IEEE Transactions on Information Theory*, 64(5):3927–3943, 2018.

[BBC+20] Marco Baldi, Alessandro Barenghi, Franco Chiaraluce, Gerardo Pelosi, and Paolo Santini. LEDAcrypt website, 2020.

[Ber01] Daniel J Bernstein. Multidigit multiplication for mathematicians. *Advances in Applied Mathematics*, pages 1–19, 2001.

[BHS80] Jon Louis Bentley, Dorothea Haken, and James B Saxe. A general method for solving divide-and-conquer recurrences. *ACM SIGACT News*, 12(3):36–44, 1980.

[BMVT78] Elwyn Berlekamp, Robert McEliece, and Henk Van Tilborg. On the inherent intractability of certain coding problems (corresp.). *IEEE Transactions on Information Theory*, 24(3):384–386, 1978.

[CA69] Stephen A Cook and Stal O Aanderaa. On the minimum computation time of functions. *Transactions of the American Mathematical Society*, 142:291–314, 1969.

[Can89] David G Cantor. On arithmetical algorithms over finite fields. *Journal of Combinatorial Theory, Series A*, 50(2):285–300, 1989.

[CCK21] Ming-Shing Chen, Tung Chou, and Markus Krausz. Optimizing bike for the intel haswell and arm cortex-m4. *Cryptology ePrint Archive*, 2021.

[CJRR99] Suresh Chari, Charanjit S Jutla, Josyula R Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In *Advances in Cryptology—CRYPTO'99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19*, pages 398–412. Springer, 1999.

[CT65] James W Cooley and John W Tukey. An algorithm for the machine calcula-

tion of complex fourier series. *Mathematics of computation*, 19(90):297–301, 1965.

[FO13] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of cryptology*, 26:80–101, 2013.

[For65] G David Forney. Concatenated codes. 1965.

[Gil52] Edgar N Gilbert. A comparison of signalling alphabets. *The Bell system technical journal*, 31(3):504–522, 1952.

[GM10] Shuhong Gao and Todd Mateer. Additive fast fourier transforms over finite fields. *IEEE Transactions on Information Theory*, 56(12):6265–6272, 2010.

[ISW+03] Yuval Ishai, Amit Sahai, David A Wagner, et al. Private circuits: Securing hardware against probing attacks. In *CRYPTO*, volume 2729, pages 463–481. Springer, 2003.

[KHF+19] Paul Kocher, Jann Horn, Anders Fogh, , Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: Exploiting speculative execution. In *40th IEEE Symposium on Security and Privacy (S&P'19)*, 2019.

[KO62] Anatolii Alekseevich Karatsuba and Yu P Ofman. Multiplication of many-digital numbers by automatic computers. In *Doklady Akademii Nauk*, volume 145, pages 293–294. Russian Academy of Sciences, 1962.

[LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S Han. Novel polynomial basis and its application to reed-solomon erasure codes. In *2014 ieee 55th annual symposium on foundations of computer science*, pages 316–325. IEEE, 2014.

[LCK+18] Wen-Ding Li, Ming-Shing Chen, Po-Chun Kuo, Chen-Mou Cheng, and Bo-Yin Yang. Frobenius additive fast fourier transform. In *Proceedings of the 2018 ACM International Symposium on Symbolic and Algebraic Computation*, pages 263–270, 2018.

[LSG+18] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. Meltdown: Reading kernel memory from user space. In *27th USENIX Security Symposium (USENIX Security 18)*, 2018.

[MAB+18] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loıc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming quasi-cyclic (hqc). *NIST PQC Round*, 2(4):13, 2018.

[MS77] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error correcting codes*, volume 16. Elsevier, 1977.

[Mul54] David E Muller. Application of boolean algebra to switching circuit design and to error detection. *Transactions of the IRE professional group on electronic computers*, (3):6–12, 1954.

[PR13] Emmanuel Prouff and Matthieu Rivain. Masking against side-channel attacks: A formal security proof. In *Advances in Cryptology–EUROCRYPT 2013: 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings 32*, pages 142–159. Springer, 2013.

[Ree53] Irving S Reed. A class of multiple-error-correcting codes and the decoding scheme. Technical report, Massachusetts Inst of Tech Lexington Lincoln Lab, 1953.

[RS60] Irving S Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the society for industrial and applied mathematics*, 8(2):300–304, 1960.

[Sin64] Richard Singleton. Maximum distance q-nary codes. *IEEE Transactions on Information Theory*, 10(2):116–118, 1964.

[SM15] Tobias Schneider and Amir Moradi. Leakage assessment methodology: A clear roadmap for side-channel evaluations. In *Cryptographic Hardware and Embedded Systems–CHES 2015: 17th International Workshop, Saint-Malo, France, September 13-16, 2015, Proceedings 17*, pages 495–513. Springer, 2015.

[Sta10] François-Xavier Standaert. Introduction to side-channel attacks. *Secure integrated circuits and systems*, pages 27–42, 2010.

[Var57] Rom Rubenovich Varshamov. Estimate of the number of signals in error correcting codes. *Docklady Akad. Nauk, SSSR*, 117:739–741, 1957.

[vDHL17] Joris van Der Hoeven and Robin Larrieu. The frobenius fft. In *Proceedings*

of the 2017 ACM on International Symposium on Symbolic and Algebraic Computation, pages 437–444, 2017.

[WZ88] Yao Wang and Xuelong Zhu. A fast algorithm for the fourier transform over finite fields and its vlsi implementation. *IEEE Journal on Selected Areas in Communications*, 6(3):572–577, 1988.

A | Appendix A: execution times

WS	Shift&Add	Karatsuba	CT Shift&Add	FAFFT
0	848	66777	7791	3062
1	848	67747	7791	3688
2	851	69653	7791	4404
3	856	72537	7791	5146
4	862	76345	7791	5892
x86	39	34	246	530

Table A.1: Execution times in kcycles for multiplication on ARM (above) and x86 (below)

WS	`reference` (R3)	`sparsemul` (R3)	`reference` (R4)	`sparsemul` (R4)	`fastmul`
0	2726	18920	136262	18965	8834
1	2748	18937	138227	18991	10107
2	2799	18986	142093	19049	11595
3	2886	19064	147971	19154	13189
4	3035	19196	155762	19324	14856
x86	148	604	157	667	1212
0	1942	9010	68750	8915	5043
1	1950	9018	69733	8926	5677
2	1965	9027	71657	8941	6409
3	1989	9045	74568	8958	7176
4	2024	9071	78415	8998	7958
x86	219	370	185	396	757

Table A.2: Execution times in kcycles for encryption and decryption

WS	`reference` (R3)	`sparsemul` (R3)	`reference` (R4)	`sparsemul` (R4)	`fastmul`
0	3913	20105	137873	20571	10441
1	3958	20148	139835	20643	11763
2	4079	20261	143631	20783	13333
3	4289	20461	149685	21051	15087
4	4626	20787	157722	21473	16994
x86	216	667	220	727	1278
0	5923	29182	206691	29557	15546
1	5974	29230	209618	29634	17504
2	6110	29361	215263	29795	19804
3	6341	29578	224238	30084	22326
4	6724	29926	236118	30540	25015
x86	440	1041	407	1145	2041

Table A.3: Execution times in kcycles for encapsulation and decapsulation

List of Figures

2.1 Butterfly unit 13
2.2 A 2-layer Truncated FFT 15
2.3 A size-8 FFT 15
2.4 Power consumption of the RSA Square and Multiply algorithm 23

3.1 Expansion of the basis conversion algorithm for $f(x) = x^6$ 32
3.2 Basis conversion, $n < 8$ 33

4.1 Multiplication execution times on ARM (left) and x86 (right) 38
4.2 Encryption/decryption execution times on ARM 39
4.3 Key encapsulation/decapsulation execution times on ARM 39

List of Tables

2.1 Parameters of HQC 27

3.1 Memory consumption (in bytes) 29

4.1 Wait states and corresponding frequencies 37

A.1 Execution times in kcycles for multiplication on ARM (above) and x86 (below) 47

A.2 Execution times in kcycles for encryption and decryption 47

A.3 Execution times in kcycles for encapsulation and decapsulation 48

List of Algorithms

1 Basis conversion from monomial to *novelpoly* 12
2 Lin-Chung-Han FFT 13
3 Frobenius Additive FFT Multiplication 16

4 Sparse polynomial multiplication 30
5 Constant-time monomial multiplication 31
6 Encoding, as a vector-matrix product 33
7 Keygen, masked version 34
8 Encrypt($\mathcal{K}_{pub}$, m), masked version 35
9 Decrypt($\mathcal{K}_{pri}$, c), masked version 35

Listings

2.1 A non constant-time password checker 21
2.2 RSA square and multiply 23

Acknowledgements

Here you might want to acknowledge someone.